

卡尔曼滤波高维计算瓶颈笔记

为什么课本里的公式依然优雅，而实现却会迅速变贵

2026 年 3 月 31 日

目录

1 为什么这件事重要	2
2 高层逻辑	3
2.1 黑盒视角	3
2.2 核心直觉	3
3 贯穿全文的两个例子	3
3.1 小例子：一辆直线运动的小车	3
3.2 大例子：网格上的温度场	4
4 所谓“计算灾难”到底从哪里来	4
4.1 存储成本	4
4.2 预测成本	4
4.3 更新成本	4
5 一个快速数量级检查	5
6 为什么解法取决于具体 benchmark	5
7 Lorenz-96 论文实际上在做什么	6
7.1 把几种方法逐个拆开	7
7.2 理论、代码和实验结果怎么对齐	9
8 五位学生的群聊讨论	12
9 工程上通常怎么做	13
9.1 利用结构	13
9.2 使用低秩思想	14

1 为什么这件事重要	2
9.3 改变线性代数表达，而不是改变目标	14
9.4 代价与权衡	14
10 一个简短总结	14
A 名词定义表	14
B 群聊人物角色卡	15
C 经典例子	16
C.1 经典的一维匀速跟踪例子	16
C.2 经典的大状态例子：网格数据同化	17
C.3 benchmark 相关的经典对照：Lorenz-63 与 Lorenz-96	17
D Lorenz-96 相关的一组一手论文	18
E Lorenz-96 的代码抓手与复现实务	19
F Lorenz-96 的补充消融与表格摘录	20
G 详细推导步骤	22
G.1 为什么稠密协方差预测是 $O(n^3)$	22
G.2 为什么更新步骤也可能变成立方成本	23
G.3 为什么小例子会让人觉得几乎没成本	24

1 为什么这件事重要

卡尔曼滤波在初学阶段看起来往往非常便宜。课堂上最常见的状态只有二维：位置和速度。矩阵一行就能写下，更新步骤甚至可以手算。这是它最友好的样子。

但工程中的版本完全不同。在天气预报、大规模传感器网络、图像重建、或者现代模型参数跟踪里，状态维度可能是几千、几万，甚至更高。此时，卡尔曼滤波遇到的障碍已经不再是“预测加校正”这个思想本身，而是要把一个稠密协方差矩阵随时间一直带着走。

如果你想先看名词速查表，请直接跳到[附录 A](#)；如果你想先看计算复杂度的详细推导，请跳到[附录 G.1](#)；如果你更想先看一个可以手算的经典小例子，请跳到[附录 C.1](#)。

老师：高维时，卡尔曼滤波并不是“理论失效”，而是“计算成本失控”。

学生 A：所以问题不在公式错了，而在于公式里背着一个非常大的矩阵？

老师：正是这样。状态估计只是一个向量，但不确定性是一个记录两两耦合关系的矩阵。真正的账单，主要来自这个矩阵。

2 高层逻辑

2.1 黑盒视角

标准线性高斯卡尔曼滤波，输入上需要四样东西：

- 上一步的状态估计；
- 状态转移模型；
- 新到来的观测；
- 以及过程噪声与测量噪声的不确定性描述。

然后它做两件事：

1. **预测**。把旧估计和旧不确定性往前推进一步。
2. **校正**。比较“预测观测”和“真实观测”，再用 Kalman 增益决定向数据靠拢多少。

输出不是单独一个均值，而是一对对象：

$$(\hat{x}_{t|t}, P_{t|t}),$$

其中 $\hat{x}_{t|t}$ 是当前最优估计， $P_{t|t}$ 记录此刻还剩下多少不确定性。

2.2 核心直觉

均值向量很容易存。真正难存的是协方差矩阵。若状态维度为 n ，那么

$$P_{t|t}$$

就有 n^2 个元素。一个稠密协方差矩阵的含义是：

我不仅要记录每个坐标各自有多不确定，还要记录每个坐标与其他所有坐标之间如何一起波动。

这在统计上很有价值，但也正是高维场景下成本爆炸的来源。

3 贯穿全文的两个例子

3.1 小例子：一辆直线运动的小车

最经典的教科书模型是

$$x_t = \begin{bmatrix} p_t \\ v_t \end{bmatrix}, \quad F = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}, \quad H = \begin{bmatrix} 1 & 0 \end{bmatrix}.$$

这个例子非常适合教学，因为它能把核心思想单独拎出来。传感器只测位置，但滤波器能同时估计位置和速度。完整的数值推演见[附录 C.1](#)。

3.2 大例子：网格上的温度场

现在把二维状态换成一个 100×100 的温度网格，则状态维度变成

$$n = 10,000.$$

这时滤波器仍然想维护一个

$$P \in \mathbb{R}^{10000 \times 10000}$$

的协方差矩阵。这个单独的稠密矩阵就有

$$10,000^2 = 100,000,000$$

个元素。若使用双精度浮点数，仅这一份矩阵就大约需要 0.8 GB 内存。实际代码中往往还要存其他矩阵和临时量，因此真实内存开销通常更大。

学生 B：一份矩阵就快到 1GB 了，那么后面的矩阵乘法和分解才是真正的大头。

老师：是的。数学形式仍然没变，但机器必须为稠密线性代数付出很高代价。

4 所谓“计算灾难”到底从哪里来

4.1 存储成本

对于稠密状态协方差

$$P_{t|t} \in \mathbb{R}^{n \times n},$$

原始存储成本就是 $O(n^2)$ 。

这件事之所以严重，是因为存储并不是一个孤立问题。一旦矩阵是稠密的，后续每一步都必须搬运、组合、分解这个稠密对象。内存压力和算力压力会一起上升。

4.2 预测成本

协方差预测步骤为

$$P_{t+1|t} = F P_{t|t} F^\top + Q.$$

如果 F 和 $P_{t|t}$ 都是稠密的 $n \times n$ 矩阵，那么主要工作就是稠密矩阵乘法，其成本为 $O(n^3)$ 。详细逐步推导见[附录 G.1](#)。

4.3 更新成本

创新协方差为

$$S_t = H P_{t|t-1} H^\top + R.$$

若观测维度为 m ，则在稠密情形下，构造并求解 S_t 的成本大约为

$$O(mn^2 + m^3).$$

若 m 与 n 同阶，那么它依然表现为 $O(n^3)$ 级别。

最核心的结论其实很朴素：

只有当问题本身带有小规模、稀疏、对角、分块可分，或者其他可利用结构时，课本上的卡尔曼滤波才会“看起来很轻松”。

5 一个快速数量级检查

状态维度 n	稠密 P 的元素数	单个稠密 P 的内存	稠密三次复杂度量级
10^2	10^4	约 0.08 MB	10^6
10^3	10^6	约 8 MB	10^9
10^4	10^8	约 800 MB	10^{12}
10^5	10^{10}	约 80 GB	10^{15}

最后一列并不是精确运行时间，只是一个数量级信号。它说明“高维计算灾难”并不是修辞，而是当 n 变大后，三次复杂度会迅速成为主角。

6 为什么解法取决于具体 benchmark

并不是所有“很贵的滤波问题”，都贵在同一个地方。这也正是为什么一个真正有用的解法，几乎总要依赖你面前的具体计算例子。一个很好的第一轮判断，是先问四个问题：

1. 动力学和观测到底是近似线性的，还是明显非线性的？
2. 状态是低维、中等规模，还是已经很高维？
3. 系统耦合更像是全局稠密，还是大多局部、稀疏？
4. 观测是全量的，还是只覆盖局部和部分分量？

光是这四个问题，就已经能把若干本质上不同的世界区分开。

世界一：线性，而且规模还不算太大。 如果模型基本线性，状态维度也没有大到失控，那么经典 Kalman filter 仍然就是最合适的对象。此时真正需要关心的，往往是数值稳定性，而不是方法与问题之间的错配。因此平方根形式、Joseph 更新、信息形式，更多是在打磨一个本来就适合该问题的方法。

世界二：非线性，但维度仍然不高。 这里的经典 benchmark 是 Lorenz-63：状态只有三个变量，但系统强非线性，而且对初值非常敏感。在这个世界里，主要障碍并不是协方差存不下，而是“局部线性化到底靠不靠谱”。所以你才会认真比较 EKF、UKF、粒子滤波。真正的痛点主要来自非线性和非高斯性，而不是维护一个 10000×10000 的协方差矩阵。

世界三：非线性、混沌，而且已经有相当维度。 这时 Lorenz-96 更有代表性。状态可以是 40 维、80 维，甚至更高，并且常常带有环状局部耦合。到了这里，问题的性格就变了。你仍然有非线性，但暴力传播协方差已经既昂贵又脆弱。所以带局部化和 inflation 的集成卡尔曼滤波会显得非常自然：它保留了随流形变化的不确定性，又不坚持去维护一个“唯一精确的稠密协方差”。

世界四：高维，但线性结构本身很好用。 有些大系统并不是主要败在非线性，而是更像“规模大，但结构清楚”。例如 PDE 离散化、网络模型、局部传感器融合等，往往带有稀疏转移、带状耦合或分块结构。此时真正有效的动作，未必是集成方法；有时更关键的是稀疏线性代数、降阶模型，或者分块协方差近似。

更大的结论其实很简单：

我们并不是在抽象地解决“那个高维 Kalman 问题”；我们是在解决它某个非常具体的几何和计算实例。

如果想看一个更直接的 benchmark 对照，请跳到[附录 C.3](#)。

7 Lorenz-96 论文实际上在做什么

如果真的去读 Lorenz-96 数据同化方面的论文，会发现一个相当稳定的图景。社区并不是在想办法把“完整的稠密 Kalman filter”原样救回来；真正的主线是换一种表示不确定性的方式。

主流解法栈。 对 Lorenz-96 这类问题，文献中的默认答案通常是：

1. 用 *ensemble* 代替完整协方差传播；
2. 做局部分析，而不是一次全局稠密更新；
3. 用 *inflation* 对抗 ensemble 过度自信；
4. 把 localization 和 inflation 的调节当成方法本体的一部分，而不是事后补丁。

这个逻辑在 Ott 等 (2002) 和 Hunt、Kostelich、Szunyogh (2005) 里已经非常清楚。它们真正的关键动作并不只是“少采样一点”，而是更结构性地把一个巨大的协方差对象拆成许多低维局部问题，并让这些问题并行求解。

为什么这特别适合 Lorenz-96。 Lorenz-96 当然是非线性的、混沌的，但它还有一个很重要的几何特征：耦合基本是局部的。每个分量主要和附近的分量相互作用。这样一来，全局稠密协方差既昂贵，又在某种意义上显得浪费。对于 Lorenz-96，localization 不是纯粹的算力技巧，它也是在让滤波器的表示方式更贴合模型本身的几何结构。

为什么 inflation 总是反复出现。 当 ensemble 很小的时候, 样本协方差天然就是秩亏的, 而且往往会显得过分自信。于是滤波器会系统性低估不确定性, 进一步导致不稳定, 甚至直接 divergence。这就是为什么 Lorenz-96 论文里 inflation 一直反复出现。Luo 和 Hoteit (2012) 在 40 维 Lorenz-96 上加入 residual nudging 来改善稳定性, 而 Attia 和 Constantinescu (2018) 则更进一步, 把 localization 半径和 inflation 因子的优化直接纳入方法设计。

更新的论文在改什么。 近年的工作通常保留同样的 benchmark 逻辑, 但尝试减少调参负担, 或者处理更强的非线性。Choi 和 Lee (2024) 用 spectrum smoothing 来缓解采样误差, 在 Lorenz 模型上它等效地诱导出空间变化的 localization 和 inflation。Bao、Zhang、Zhang (2023) 提出 Ensemble Score Filter, 去应对高度非线性、高维的情形。Robinson 和 Grooms (2020) 讨论了 medium nonlinearity 下的 hybrid particle-ensemble Kalman filter。Bocquet 等 (2024) 则表明, 在 Lorenz-96 上, 一个学到的 analysis map 有可能在只传播单个 forecast state 的情况下, 逼近一个调得很好的 ensemble filter。Ait-El-Fquih 和 Hoteit (2026) 则直接去改 localization 本身, 试图让“局部性”内生到分析步里, 而不是靠后处理的人为裁剪。

7.1 把几种方法逐个拆开

LETKF / localized EnKF: 先把“大问题”切成很多“小问题”。 它要解决的核心失败模式, 是: 全局稠密协方差太大, 而小 ensemble 给出的全局样本协方差又太假。LETKF 的输入是 forecast ensemble、局部观测和观测误差; 第一步, 把每个网格点附近的一小块状态和观测取出来; 第二步, 不在全状态空间做分析, 而是在 ensemble 张成的低维权重空间里求一次小型 Kalman 更新; 第三步, 把每个局部分析重新拼回全局 analysis ensemble。关键推导关系只有一句: 分析协方差不再在 n 维状态空间里求, 而是在 $N_e - 1$ 维 ensemble 子空间里求, 所以最重的线性代数从“状态维度主导”改成了“ensemble 尺寸主导”。这特别适合 Lorenz-96, 因为它本来就是局部耦合环。代价则是: 你必须自己选择局部窗口、观测截断和 inflation; locality 选坏了, 误差相关就会被切断得过头。Ott 等 (2002) 与 Hunt、Kostelich、Szunyogh (2005) 是这条线的基础论文。

Residual nudging: 不改整个滤波器, 只在“分析残差太离谱”时拉一把。 它要解决的问题不是复杂度, 而是稳定性: 有些时候 EnKF 给出的 analysis 在观测空间里离真实观测还是太远, 这通常意味着滤波器已经过度自信。输入是普通 EAKF 的 analysis mean、观测 y 和一个“允许多大残差”的阈值; 第一步, 先算 analysis residual $r^a = y - Hx^a$; 第二步, 如果 $\|r^a\|$ 太大, 就把 analysis mean 和一个“观测反演”状态 x^o 做凸组合; 第三步, 只改 mean, 不改协方差, 继续往后跑。最关键的关系是

$$\tilde{x}^a = cx^a + (1 - c)x^o,$$

其中 c 选到刚好把残差压回阈值附近。直觉上, 这相当于在滤波器快要偏航时, 强行要求“至少别在观测空间里说得太离谱”。它在 Lorenz-96 上有效, 是因为小 ensemble 场景下

divergence 很常见；但代价是它主要修的是稳定性，不会神奇地提升所有情形下的最优精度。Luo 和 Hoteit (2012) 给出了完整分析和 40 维 Lorenz-96 实验。

Adaptive inflation / localization: 把“经验调参”变成在线优化。 它针对的失败模式是：固定的 inflation 因子和 localization 半径往往既不随时间变，也不随空间位置变，而 Lorenz-96 的误差结构明明是时变、空变的。输入是 forecast ensemble、当前观测和一个设计目标；第一步，把 inflation 向量 λ 或 localization 半径向量 L 当成待优化变量；第二步，定义目标函数为 analysis 后验协方差的迹，再加一点正则项，避免解退化到边界；第三步，用梯度法算出本轮最优 λ 或 L ，再做 analysis。理论上最关键的一句是：它不直接最小化当前 RMSE，而是最小化“这一步分析后还剩多少总不确定性”，即后验方差和。Lorenz-96 上之所以自然，是因为不同位置的不稳定程度本来就不同。代价则是：每一步 assimilation 前多了一层优化，数值实现复杂得多。Attia 和 Constantinescu (2018) 在 two-layer Lorenz-96 上系统做了这件事。

Spectrum smoothing: 先改 ensemble 的频谱，再让 ETKF 去更新。 它解决的是小样本采样误差。经典 localization 直接在物理空间截远程相关；spectrum smoothing 则先去频域修 ensemble。输入是 prior ensemble；第一步，对 ensemble perturbations 做 Fourier 变换，估计 mean power spectrum；第二步，用卷积核把频谱抹平；第三步，按新的平滑频谱重新缩放各个波数上的幅值；第四步，再回到 ETKF 的普通 analysis。最重要的理论直觉来自 Wiener-Khinchin：频谱和自相关是一一对应的，所以把频谱修得更像湍流系统应有的平滑衰减，自相关估计也会更合理。它对 Lorenz-96 有效，前提是系统已经进入足够明显的湍流谱区；若维度太低或谱本来就不明显，这个动作收益有限。Choi 和 Lee (2024) 明确指出，在 Lorenz-96 上它等效地产生了空间不均匀的 localization / inflation。

Ensemble Score Filter: 不用高斯线性更新，改成在伪时间里逐步注入 likelihood。 它瞄准的是“高维 + 强非线性观测”这一档问题，此时 LETKF 常常高度依赖手工调 localization / inflation。输入是 prior state ensemble、观测、观测模型；第一步，用扩散模型的 score 表示 prior density，而不是只存样本协方差；第二步，把 likelihood 的梯度逐步加入 posterior score；第三步，在伪时间上解一个反向采样过程，把 prior 推到 posterior。核心关系可以压缩成一句：

$$\nabla \log p(x | y) \approx \nabla \log p(x) + h(\tau) \nabla \log p(y | x),$$

也就是把“先验长什么样”和“观测想把你往哪拉”这两股力量在伪时间里慢慢混合。Lorenz-96 上它能工作，是因为它直接处理 posterior 的非高斯形状，而不是把一切都压成协方差。代价是：算法栈更重，里面有扩散时间步、噪声日程、反向求解器等新超参数。Bao、Zhang、Zhang (2023) 是这一方向的代表。

Hybrid particle-EnKF: 先用一点 particle filter，把先验拉得更像高斯，再交给 EnKF。

它解决的是 medium nonlinearity：先验已经明显非高斯，但 posterior 还不至于太离谱。

输入是 ensemble、观测和一个 ESS 目标；第一步，把 likelihood 拆成两段

$$p(y | x) = p(y | x)^\alpha p(y | x)^{1-\alpha},$$

先用第一段做 particle filter 更新；第二步，根据 ESS 阈值选 α ，避免 particle weights 直接塌缩；第三步，对重采样后的 ensemble 做随机正交变换打破退化；第四步，再用 ESRF 吃掉剩下那段 likelihood。直觉上，它不是让 PF 或 EnKF 单打独斗，而是让 PF 先把最明显的非高斯性削掉，再让 EnKF 在一个“更像高斯”的中间分布上收尾。Lorenz-96 上有效的条件是：确实存在“先验歪，但 posterior 还行”的窗口。代价是参数更多，而且 ensemble 不够大时，PF 这半步依然可能白做。Robinson 和 Grooms (2020) 讨论得很清楚。

Learned analysis map / DAN：把“误差该往哪里修”直接学成一个算子。 它要回答的尖锐问题是：能不能不显式传播 ensemble，也大致抓住 EnKF 真正在做的那部分事。输入是单个 forecast state 和当前观测；第一步，在 twin experiment 上训练一个 analysis operator a_θ ；第二步，在线时直接把 forecast 和观测送进 a_θ ；第三步，输出 analysis state。核心理论直觉不是“神经网络比贝叶斯更高明”，而是：对 Lorenz-96 这类混沌系统，真正重要的误差方向往往集中在少数不稳定方向里，而这些方向和当前状态有稳定映射关系。于是，一个学到的 analysis map 可能在不显式携带 ensemble 的情况下，近似恢复这些方向。代价是它高度 benchmark-driven，需要训练，而且泛化边界必须单独验证。Bocquet 等 (2024) 给出了最有说服力的一组结果。

7.2 理论、代码和实验结果怎么对齐

理论上最该记住的，不是长推导，而是四个“降难度动作”。

1. LETKF 把分析步从状态空间搬到 ensemble 权重空间，本质是把“大矩阵问题”改写成“小矩阵问题”。
2. residual nudging 在 analysis 残差过大时做凸组合，本质是给滤波器加一道“观测空间合理性约束”。
3. hybrid PF-EnKF 把 likelihood 拆成两段，本质是先修先验形状，再做近高斯更新。
4. score filter 用 posterior score 代替 posterior covariance，本质是把“分布几何”作为一等公民来表示。

代码层面真正会写成什么。

- **LETKF / adaptive EnKF**：主循环基本是“forecast → 取局部窗口 → 小规模优化/更新 → 拼回全局”。DATeS 的 adaptive_EnKF.py 直接把 inflation 和 localization 写成有界优化问题，并在优化后做 moving average 平滑，这和论文里的 OED 设计完全对应。

- **EnSF**: [官方仓库](#) 把实现拆成 NoiseSchedule、ScoreRep、ReverseSampler 三层；`fine_tune_EnSF/results/final_rmse_EnSF.csv` 与 `run_all_EnSF/param_EnSF.csv` 则直接暴露了 Nt_SDE 、 ϵ_a 、 ϵ_b 、ensemble size 等核心旋钮，因此论文里的最优组合可以被代码逐项核对。
- **spectrum smoothing**: 代码并不复杂，关键是多了一步 Fourier-domain rescaling；也就是说，ETKF 主体不变，只是在每次 assimilation 之前先做一次频谱预处理。
- **hybrid PF-EnKF**: [Robinson](#) 和 [Grooms](#) 的公开代码/数据仓库 显示，最关键的三个旋钮就是 inflation、localization radius 和 ESS threshold；这正对应论文里“likelihood splitting + 防塌缩”的设计。

先说明一个阅读原则。 下面这些结果表都是真实论文中的具体数字，但不同论文的绝对 *RMSE* 不能直接横比，因为状态维度、观测算子、观测噪声、assimilation interval 和是否用 two-layer Lorenz-96 都不一样。所以最可靠的读法是：先在同一张表、同一个设置内比较谁更好，再看跨论文时“哪种失败模式被修掉了”。

表一：residual nudging 在同一论文内的消融最清楚。

场景	原始 EAKF 最优 time-mean RMSE	EAKF-RN 最优 time-mean RMSE	额外现象
全观测	0.5605 ($l_c = 0.1, \lambda = 1.10$)	0.5586 (同一组参数)	在 ($l_c = 0.4, \lambda = 1.05$) 处，EAKF 约 3.3，而 EAKF-RN 约 1.6； ($l_c = 0.3, \lambda = 1.25$) 处原始 EAKF 发散，RN 未发散
1/2 观测	0.9515 ($l_c = 0.1, \lambda = 1.20$)	0.9474 ($l_c = 0.1, \lambda = 1.20$)	原始 EAKF 在 30 组参数里发散 24 组； EAKF-RN 发散 0 组
1/4 观测	1.9908 ($l_c = 0.1, \lambda = 1.05$)	1.9860 ($l_c = 0.1, \lambda = 1.10$)	原始 EAKF 在 30 组参数里发散 17 组； EAKF-RN 发散 0 组
1/8 观测	2.9619 ($l_c = 0.1, \lambda = 1.00$)	2.9556 ($l_c = 0.1, \lambda = 1.00$)	两者都稳定，但 RN 仍略优

这个表非常说明问题：residual nudging 对“最优指标”的提升通常不大，但对稳定性的提升很大。换句话说，它更像是让滤波器“不容易死”，而不是把一个本来就调得很好的点继续大幅推低。

表二：spectrum smoothing 的收益强依赖湍流强度。

Lorenz-96 设置	观测/ensemble	基线 ETKF	ETKF + spectrum smoothing
弱湍流 $F = 4$	100% 观测, $K = 40$	0.0027	0.0043
中等湍流 $F = 8$	25% 观测, $K = 10$	4.5024	3.0801
中等湍流 $F = 8$	50% 观测, $K = 30$	3.2940	0.2186
强湍流 $F = 16$	100% 观测, $K = 40$	4.3081	0.3122

这组结果的关键信息很明确：当 Lorenz-96 的谱已经有足够明显的湍流衰减结构时，频谱平滑能显著减少采样误差；但在弱湍流、低维或频谱本来不够“干净”的区间，它不保证稳赢。论文甚至明确说，40 维标准 L96 上改进很边际，所以这不是一个“无脑加就好”的万能模块。

表三：adaptive tuning、score filter 和 learned analysis 的代表性结果。

论文	同一论文内的可比设置	结果	最值得记住的点
Attia & Constantinescu (2018)	two-layer Lorenz-96; 固定空间/时间上的 benchmark 调参网格	最优 benchmark 为 RMSE = 0.170 , 对应 $\lambda = 1.5$ 、 $l = 0.5$	先把“经验参数”调到位，再谈 adaptive; adaptive 的意义是减少人工扫参
Bao, Zhang, Zhang (2023)	100 维上做超参数微调, 再迁移到 10^6 维 Lorenz-96	LETKF 最优组合: $(1.1, 4), (1.0, 2), (1.1, 3)$; EnSF 最优组合: $(\epsilon_a, \epsilon_b) = (0.5, 0.025), (0.6, 0.025), (0.5, 0.025)$; 在 10^6 维时 EnSF GPU 单步约 5s , LETKF 约 300s	EnSF 的优势不仅是精度, 更是超参数更平稳、GPU 友好
Bocquet 等 (2024)	40 维 L96; 比较 classical DA 与 learned analysis map	EnKF-N ($N_e = 20$): 0.191; EnKF-N ($N_e = 40$): 0.179 ; DAN ($N_e = 1, N_f = 40$): 0.191; DAN ($N_e = 1, N_f = 100$): 0.185; 3D-Var: 0.40	只传播单个 forecast state 的 DAN, 已经接近 well-tuned EnKF

表四：hybrid PF–EnKF 的结论更像 “边界条件说明书”。

设置	最优参数（论文报告）	相对 ESRF 的收益	解释
$N = 400$ 的 multiscale Lorenz-96	纯 ESRF: $r = 0.026, L = 209$; SIR-ESRF: $r = 0.06, L = 279$; BSIR-ESRF: 更大 L	三者差异不大, hybrid 还没有明显压过 ESRF	ensemble 还不够大, PF 这一步能削掉的非高斯性有限
$N = 1200$ 的 multiscale Lorenz-96	纯 ESRF: $r = 0.015, L = 250$; SIR-ESRF: $r = 0.04, L = 316, ESS = 757$; BSIR-ESRF: $r = 0.02, L = 319, ESS = 642$	BSIR-ESRF 在 CRPS 与 RMSE 上比 ESRF、SIR-ESRF 约好 5%–10%	说明 hybrid 主要适合 medium nonlinearity, 而且往往需要更大的 ensemble 才能赢

把这些表格合起来看，会得到一个比 “方法名清单” 更有用的结论。 Lorenz-96 文献里的创新并不是在随机往滤波器上贴新模块，而是在分别修四种不同的失败模式： LETKF 修的是高维局部结构， residual nudging 修的是不稳定性， adaptive inflation/localization 修的是手工调参， spectrum smoothing 修的是采样误差的频域结构， hybrid PF–EnKF 修的是中等非高斯性， EnSF 和 DAN 则是在尝试替换 “协方差这一表示方式” 本身。

实践上的结论。 如果问题是 “Lorenz-96 到底先该试什么?”，那么文献支持下的答案仍然是：

先从带 localization 的 ensemble Kalman 方法做起，然后优先把 localization 和 inflation 调稳，再考虑更花哨的机器。

只有当观测高度非线性、ensemble 尺寸非常受限，或者你想减少大量手工调参时，更激进的方法才真正变得有吸引力。一个简洁的论文清单见[附录 D](#)。

8 五位学生的群聊讨论

要真正感受到这个瓶颈，让几种不同气质的人围着同一张桌子慢慢聊，往往比单独看一张复杂度表更有用。学生 A 到学生 E 的简短角色卡见[附录 B](#)。正文里，我们只是在听：同一组公式，落进五个人脑中，会说出怎样不同的话。

学生 A：我现在看到卡尔曼滤波，还是会先被它纸面上的平静打动。预测，校正，再预测，再校正。整套记号几乎是安静的，仿佛公式本身并不急着提醒你麻烦马上就要来了。

学生 B: 机器可没这么容易被安抚。对它来说，最醒目的不是“平静”，而是那个 $n \times n$ 的协方差矩阵：要存，要推，要乘，还要分解。只要开始认真数内存和浮点操作，屋里的气氛就变了。

学生 C: 这里有个区分，我觉得最好始终说清。在线性高斯模型下，数学仍然是精确的。真正开始变脆弱的，不是这套逻辑本身，而是我们是否还能在大规模上把它完整做出来。

学生 D: 而且一旦状态带有空间结构，协方差的含义会一下子具体起来。它不只是一个方差账本，而是在记录：这一块区域如何牵动另一块区域，这个位置如何和其余位置一起起伏。这样的记录长得非常快。

学生 E: 所以甚至在真正测 runtime 之前，几何关系就已经不对称了：状态规模是线性增长的，稠密协方差是平方增长的，最重的线性代数常常还是立方增长的。整件事的张力，大致就浓缩在这一句里。

学生 A: 100×100 的网格例子就把这种张力照得很亮。听到“一万个状态变量”时，人还容易觉得事情尚可控制；可一旦说成“协方差里有一亿个元素”，感觉立刻就不一样了。

学生 B: 对，而且那还只是存储账单。真正昂贵的地方，是代码开始算 FPF^T 、开始组装创新协方差、开始求解对应线性系统的时候。这些都不是顺手带过的小步骤，而是整次更新里最重的部分。

学生 C: 这也正是为什么教材里的小车例子总显得那么无害。因为在 $n = 2$ 时，创新协方差只是一个标量，矩阵乘法几乎没有分量，人很容易把“小尺度下的轻松”误当成“普遍的轻松”。

学生 D: 于是，实践里的问题也就悄悄换了。它不再是“卡尔曼滤波能不能写出来”。当然能。真正的问题是：哪些相关性值得保留，哪些结构可靠到足以被利用。

学生 E: 这也就是那些熟悉的出路为什么会显得很有分寸：稀疏、局部化、低秩、集成近似、顺序更新。它们不是贴在理论外面的补丁，而是在认真决定：我们到底负担得起怎样的不确定性表示。

学生 A: 我很喜欢这种说法。这样一来，整个主题就不像“理论忽然失灵”，反而更像是在精度和规模之间进行一场有节制的协商。

学生 B: 可以这么理解。若坚持处处都保留稠密而精确的表达，计算账单会非常冷酷；若能聪明地利用结构，滤波器就重新变得可操作了。

9 工程上通常怎么做

9.1 利用结构

若状态之间只有局部耦合，那么 F 、 Q ，甚至 P 本身都可能是稀疏或带状的。分块对角近似或带状近似，本质上是在主动丢弃弱的远程相关性，只保留最重要的相互作用。

9.2 使用低秩思想

有时主要不确定性实际上只活跃在一个远小于原始维度的子空间里。此时就可以只跟踪低秩因子，或者采用基于样本的集成方法，用样本协方差近似真实协方差，而不直接存整个稠密矩阵。

9.3 改变线性代数表达，而不是改变目标

平方根滤波、信息滤波、顺序测量更新、集成卡尔曼滤波等方法，都保留了“预测 + 校正”这一主线。它们真正改变的，主要是如何表示不确定性，以及如何减轻最昂贵的线性代数步骤。

9.4 代价与权衡

每条逃生路线都有代价：

- 稀疏或分块近似可以降成本，但可能遗漏远距离相关性；
- 低秩方法节省内存，但可能漏掉那些后来会变重要的不确定方向；
- 集成方法避免显式存稠密协方差，但要用 Monte Carlo 近似换取精确性；
- 顺序或因子化更新只有在观测模型确实有结构时才会真正便宜。

学生 C： 所以，并不存在那种既完全精确、又完全稠密、同时还特别便宜的高维版本？

老师： 这是对的。大规模卡尔曼滤波，核心上就是在问：你愿意相信哪种结构，足以让你放心地利用它。

10 一个简短总结

卡尔曼滤波解决的是一个非常干净的推断问题：把模型预测和新观测结合起来。在低维时，它既优雅又高效；在高维时，真正的阻碍不是思想，而是协方差矩阵。稠密协方差的存储成本是 $O(n^2)$ ，而传播和分解常常是 $O(n^3)$ 。因此，工程实践里的关键不是放弃滤波，而是重新设计不确定性的表示方式，让滤波器顺着问题的结构工作，而不是和维度正面硬碰硬。

A 名词定义表

术语	通俗含义
状态 x_t	我们真正关心、但通常不能直接完整看到的量，例如位置、速度、温度场或参数向量。

术语	通俗含义
估计 $\hat{x}_{t t}$	利用截至时刻 t 的观测后，滤波器给出的当前最佳猜测。
协方差 $P_{t t}$	描述估计有多不确定，以及不同坐标之间如何共同波动的矩阵。
过程噪声 Q	由于动力学模型不完美而引入的不确定性。
测量噪声 R	由传感器本身带来的不确定性。
创新	“预测观测”和“真实观测”之间的差。
Kalman 增益 K_t	把创新翻译成状态修正量的矩阵。
稠密矩阵	大多数元素都非零的矩阵，因此通用矩阵乘法和分解都很贵。
稀疏矩阵	有大量零元素的矩阵，因此可以利用专门算法跳过很多无效计算。
低秩近似	只保留主要变化方向的一种压缩表示。
观测维度 m	单次更新里，传感器返回多少个观测数值。
计算瓶颈	在时间或内存上占主要成本、从而让直接方法变得不现实的步骤。

B 群聊人物角色卡

人物	背景	人物性格	学术风格
学生 A	MIT 数学系本科生	对形式美很敏感，耐心，也带着一点安静的兴奋感	往往先从结构和直觉进入，再看一个漂亮的定理怎样落到具体例子里
学生 B	计算机系博士生	冷静务实，对隐藏成本尤其敏锐	在真正欣赏公式之前，先拿运行时间、内存流量和实现负担去检验它
学生 C	Stanford 数学系旁听生	克制、精确，也喜欢把边界分清楚	擅长把“概念上成立”和“计算上可行”这两层意思稳稳拆开
学生 D	PKU 访客学生	温和，直观感强，天然关心大尺度结构	常把公式重新投回网格、场、几何和空间相互作用中去理解

人物	背景	人物性格	学术风格
学生 E	THU 访客学生	沉着，善于归纳， 也很有工程感	喜欢把较长的论证压缩成设计选择，并始终把理论拴回可实现的近似

C 经典例子

C.1 经典的一维匀速跟踪例子

考虑最熟悉的两维状态模型

$$x_t = \begin{bmatrix} p_t \\ v_t \end{bmatrix}, \quad F = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}, \quad H = \begin{bmatrix} 1 & 0 \end{bmatrix}.$$

设上一步滤波结果为

$$\hat{x}_{t-1|t-1} = \begin{bmatrix} 10 \\ 1 \end{bmatrix}, \quad P_{t-1|t-1} = \begin{bmatrix} 4 & 1 \\ 1 & 1 \end{bmatrix}.$$

再取

$$Q = \begin{bmatrix} 0.2 & 0.1 \\ 0.1 & 0.1 \end{bmatrix}, \quad R = [1], \quad y_t = 12.$$

预测。 预测均值为

$$\hat{x}_{t|t-1} = F\hat{x}_{t-1|t-1} = \begin{bmatrix} 11 \\ 1 \end{bmatrix}.$$

预测协方差为

$$P_{t|t-1} = FP_{t-1|t-1}F^\top + Q = \begin{bmatrix} 7.2 & 2.1 \\ 2.1 & 1.1 \end{bmatrix}.$$

校正。 预测观测为

$$H\hat{x}_{t|t-1} = 11,$$

所以创新为

$$\tilde{y}_t = y_t - H\hat{x}_{t|t-1} = 1.$$

创新协方差为

$$S_t = HP_{t|t-1}H^\top + R = 7.2 + 1 = 8.2.$$

Kalman 增益为

$$K_t = P_{t|t-1}H^\top S_t^{-1} = \frac{1}{8.2} \begin{bmatrix} 7.2 \\ 2.1 \end{bmatrix} \approx \begin{bmatrix} 0.8780 \\ 0.2561 \end{bmatrix}.$$

更新后的状态估计为

$$\hat{x}_{t|t} = \hat{x}_{t|t-1} + K_t \tilde{y}_t \approx \begin{bmatrix} 11.8780 \\ 1.2561 \end{bmatrix}.$$

更新后的协方差为

$$P_{t|t} = P_{t|t-1} - K_t S_t K_t^\top \approx \begin{bmatrix} 0.8780 & 0.2561 \\ 0.2561 & 0.5622 \end{bmatrix}.$$

这个例子之所以适合课堂，是因为它足够小，可以手算完。但它也容易让人误判高维情形的真实代价，因为这里 $n = 2$ ，一切都太小了。

C.2 经典的大状态例子：网格数据同化

考虑一个 100×100 的温度网格，状态维度为 $n = 10,000$ ，并假设协方差是稠密的。

- 协方差共有 10^8 个元素；
- 单个稠密协方差矩阵在双精度下约需 0.8 GB；
- 稠密协方差预测的计算量量级约为 $O(n^3) \approx 10^{12}$ 。

这正是大规模数据同化中普遍采用局部化、集成近似、降阶模型等结构化方法的经典原因。滤波方程本身仍然成立，但稠密实现已经不现实。

C.3 benchmark 相关的经典对照：Lorenz-63 与 Lorenz-96

把两个经常被放在同一个“大类标签”里的 benchmark 并排看，会很有帮助。它们都属于“非线性滤波”语境，但真正考验的方法弱点并不一样。

Lorenz-63。 这个系统只有三个状态变量，所以维度并不是主要敌人。真正困难的是强非线性，以及对初始条件的敏感依赖。在这种情况下：

- 若局部线性化还可信，EKF 可能勉强可用；
- 由于维度很小，UKF 的 sigma-point 传播并不算贵；
- 粒子方法也还没有被维度压垮到完全不可谈。

因此，这里的主要问题首先是非线性。

Lorenz-96。 这个系统则常被用作更高维的数据同化 benchmark。状态可以是 40 维、80 维，甚至更多，而且通常带有局部耦合。此时困难的性质就不同了：

- 暴力的 EKF 式协方差传播会变得很贵；
- 稠密协方差表示越来越显得不自然，因为真正重要的相关性通常是局部的、随流而变的；

- 带局部化和 inflation 的集成方法，会更贴合这个问题本身的几何结构。

因此，这里的问题是非线性加上可扩展的不确定性表示。

真正的分界。 Lorenz-63 教我们：即使维度很小，天真的线性化也可能失败。Lorenz-96 教我们：一旦维度上来，核心问题就不再只是“线性还是非线性”，而会变成“哪种不确定性表示最符合这个系统的结构”。

D Lorenz-96 相关的一组一手论文

论文	对本笔记最重要的启发
Ott 等 (2002), <i>A Local Ensemble Kalman Filter for Atmospheric Data Assimilation</i>	提出 local EnKF 的核心思想：每个局部区域的预报误差往往落在一个远低于全局维度的子空间里，因此可以把一次昂贵的全局更新改写成许多低维局部分析；Lorenz-96 是它的重要演示对象。
Hunt、Kostelich、Szunyogh (2005), <i>Efficient Data Assimilation for Spatiotemporal Chaos: a Local Ensemble Transform Kalman Filter</i>	把 LETKF 做成对大规模时空混沌系统真正实用的方法；论文强调的重点非常明确，就是易用性和计算速度，而不是坚持维护完整稠密协方差。
Luo 和 Hoteit (2012), <i>Ensemble Kalman filtering with residual nudging</i>	在 40 维 Lorenz-96 上说明 residual nudging 可以在小 ensemble 情况下提升精度或增强稳定性，尤其是在滤波器容易过度自信时。
Attia 和 Constantinescu (2018), <i>An Optimal Experimental Design Framework for Adaptive Inflation and Covariance Localization for Ensemble Filters</i>	说明 inflation 和 localization 不应被当成经验调参碎片，而应被视为方法中的核心可优化对象；用的是 two-layer Lorenz-96 benchmark。

论文	对本笔记最重要的启发
Choi 和 Lee (2024), <i>Sampling error mitigation through spectrum smoothing</i>	用 spectrum smoothing 来缓解采样误差；在 Lorenz-96 上，它可以理解成一种会随空间位置变化的 localization/inflation 机制。
Bao、Zhang、Zhang (2023), <i>An Ensemble Score Filter for Tracking High-Dimensional Nonlinear Dynamical Systems</i>	代表更近年的一条路线：当标准 EnKF 式高斯假设太吃力时，尝试用 score-based 表示去做高维强非线性滤波；Lorenz-96 是核心测试对象之一。
Robinson 和 Grooms (2020), <i>A hybrid particle-ensemble Kalman filter for problems with medium nonlinearity</i>	面向的是“预报明显非高斯，但纯 particle filter 又太脆弱”的中间地带；可以把它看成 Lorenz-96 动机下，EnKF 与粒子方法之间的一种折中。
Bocquet 等 (2024), <i>Accurate deep learning-based filtering for chaotic dynamics by identifying instabilities without an ensemble</i>	说明对 Lorenz-96 而言，有时甚至可以学出一个 analysis step，在不显式传播 ensemble 的情况下逼近调得很好的 EnKF，关键在于抓住真正重要的不稳定方向。
Ait-El-Fquih 和 Hoteit (2026), <i>A Structurally Localized Ensemble Kalman Filtering Approach</i>	说明直到很近的工作，重点仍然是在改进 localization，而不是抛弃它：目标是把局部性直接做进分析步里，同时减少那种 ad hoc 的人工调 localization。

E Lorenz-96 的代码抓手与复现实务

方法	代码里新增的关键模块	对实现者最重要的提示
LETKF / local EnKF	局部观测索引、局部 forecast anomalies、ensemble-space 变换矩阵、局部到全局的 scatter/gather	真正的主循环是“遍历局部窗”，不是“显式更新全局协方差”。若代码仍在存全局 P ，那它基本不是 Lorenz-96 文献里主流的写法。
Residual nudging	残差计算、阈值判定、analysis mean 的凸组合后处理	最适合作为已有 EAKF/EnKF 代码的稳定性补丁接入；它通常不要重写 covariance update。
Adaptive inflation / localization	每个 assimilation cycle 的目标函数、梯度、约束优化器与参数回填	难点不在 Kalman 更新，而在“把调参外循环写稳”。若没有梯度或停止准则，代码会非常脆。
Spectrum smoothing	FFT / iFFT、频域 rescaling、平滑谱构造、与 LETKF 分析步的衔接	这不是额外堆一个黑箱网络，而是把结构先验写进 covariance estimation。实现上最容易出错的是频域缩放和物理域回投的一致性。
Hybrid particle-EnKF	likelihood splitting、ESS 阈值求根、重采样、随机正交变换、随后 ESRF 更新	关键不是“先粒子后 EnKF”这句话，而是如何保证第一阶段不过度坍缩，使第二阶段还剩足够多的有效样本。
EnSF	公开仓库 中的 ScoreFilter/diffusion.py、ScoreFilter/utils.py、run_all_EnSF/、run_all_LETKF/	仓库把 EnSF 和 LETKF 并排组织，这很有价值：你可以直接看到 baseline 与新方法是如何在同一问题配置上做参数扫描的。L96 drift、RK4 推进、atan 观测 score、RMSE/CRPS 批量评测都已给出。
Learned analysis map	CNN analysis operator、长期轨道数据集、截断反向传播、在线单步推理接口	训练阶段像时序学习；推理阶段像局部卷积校正器。部署时的计算负担主要从线性代数转移到离线训练。

F Lorenz-96 的补充消融与表格摘录

论文	被改动的变量	论文里真正看到的现象
Luo 和 Hoteit (2012)	观测稀疏度、 l_c 、 λ 、 β 、ensemble size	在 1/2 观测场景，普通 EAKF 在 30 个参数组合里有 24 个 divergence，而 EAKF-RN 无 divergence；在 1/4 观测下，普通 EAKF 有 17/30 个 divergence，而 EAKF-RN 仍无 divergence。说明 residual nudging 的主要收益是稳定性而不是在最优参数点上大幅压低误差。
Attia 和 Constantinescu (2018)	inflation 正则系数 α 、localization 正则系数 γ 、ensemble size、观测噪声	论文先用 two-layer Lorenz-96 扫出经验 benchmark: $\lambda = 1.5, l = 0.5$ 。随后表明 adaptive inflation 在 $\alpha \approx 0.0015$ 附近落在 L-curve 肘部，能在不做大规模手工网格搜索的情况下追上 benchmark；adaptive localization 则显示 $\gamma = 0$ 也可获得良好结果，但 observation-space 版本的优化迭代更贵。
Choi 和 Lee (2024)	ensemble size K 、观测密度、localization/inflation 参数、是否做 spectrum smoothing	smoothing 的收益高度依赖“样本少或观测稀”。 $K = 10$ 或 25% 观测时，经典 ETKF 的 RMSE 会飙升到 3~4，而 smoothing 版本仍能落在 0.18~3.23 范围内；当 $K = 40$ 且 100% 观测时，两者都接近基线，改进几乎消失。结论很清楚：这不是普适 magic，而是针对采样误差的结构化补丁。
Robinson 和 Grooms (2020)	ensemble size N 、ESS 阈值、inflation r 、localization 半径 L 、是否 blur 观测	当 $N < 400$ 时，三种方法几乎不分胜负，说明瓶颈仍是 sampling error；到了 $N = 1200$ ，BSIR-ESRF 才开始稳定优于 ESRF。作者据此给出一个非常重要的判断：混合粒子法要赢过纯 EnKF，往往需要比纯 EnKF 更大的 ensemble。

论文	被改动的变量	论文里真正看到的现象
Bocquet 等 (2024)	ensemble forecast size N_e 、CNN filters N_f 、观测噪声、观测 密度、状态维度 N_x	DAN 在 $N_e = 1$ 时 aRMSE 已在 0.19~0.20 间波动；增加 filters 到 $N_f = 100$ 可到 0.185。更关键的消 融是跨维迁移：把在 $N_x = 40$ 学到的 算子直接用到 $N_x \in [20, 160]$ ， aRMSE 仍大致维持在 0.188~0.197。这说明网络学到的是 局部、不随维度暴涨而失效的结构。
Bao、Zhang、 Zhang (2023)	维度 100, 200, 10^6 、 ensemble size、SDE 步数、非线性观测、 mini-batch 大小	文中把 EnSF 的成本分析写得很明 确：存储主要是样本矩阵，单步浮 点量级与维度和 ensemble size 近 似线性增长。公开仓库也把 Nt_SDE、ensemble_size、 inflation、eps_a/eps_b 等参数 直接暴露在 CSV/Notebook 里，方 便做系统消融。

G 详细推导步骤

G.1 为什么稠密协方差预测是 $O(n^3)$

从

$$P_{t+1|t} = F P_{t|t} F^\top + Q$$

出发，其中 $F, P_{t|t}, Q \in \mathbb{R}^{n \times n}$ ，并把它们都视作稠密矩阵。

enter 先只盯住矩阵尺寸。除了最后的加法外，其余对象全是 $n \times n$ 的稠密矩阵。

$$P_{t+1|t} = F P_{t|t} F^\top + Q.$$

rw 先做前两个稠密矩阵的乘法。

$$M_1 = F P_{t|t}, \quad M_1 \in \mathbb{R}^{n \times n}.$$

一个稠密 $n \times n$ 矩阵乘另一个稠密 $n \times n$ 矩阵，操作量级为 n^3 。

rw 再把结果乘上 $F^\top$ 。

$$M_2 = M_1 F^\top, \quad M_2 \in \mathbb{R}^{n \times n}.$$

这又是一次 n^3 量级的稠密矩阵乘法。

done 最后再加上过程噪声协方差。

$$P_{t+1|t} = M_2 + Q.$$

加法本身只要 $O(n^2)$ ，远小于前面的两个三次步骤。

因此，稠密协方差预测的主导成本为

$$O(n^3) + O(n^3) + O(n^2) = O(n^3).$$

G.2 为什么更新步骤也可能变成立方成本

更新用到

$$S_t = HP_{t|t-1}H^\top + R, \quad K_t = P_{t|t-1}H^\top S_t^{-1}.$$

设 $H \in \mathbb{R}^{m \times n}$ 。

enter 先构造创新协方差。

$$S_t = HP_{t|t-1}H^\top + R.$$

rw 先计算 $HP_{t|t-1}$ 。

$$N_1 = HP_{t|t-1}, \quad N_1 \in \mathbb{R}^{m \times n}.$$

这一步的稠密乘法成本为 $O(mn^2)$ 。

rw 再乘上 $H^\top$ 。

$$N_2 = N_1H^\top, \quad N_2 \in \mathbb{R}^{m \times m}.$$

这一步成本为 $O(m^2n)$ 。当 $m \leq n$ 时，它被 $O(mn^2)$ 控住。

have 接着要对 $m \times m$ 的创新协方差做求解。

$$S_t = N_2 + R.$$

对一个稠密 $m \times m$ 矩阵做分解或求逆，成本为 $O(m^3)$ 。

done 把两块主要成本合起来。

$$\text{update cost} \approx O(mn^2 + m^3).$$

当 m 与 n 同阶时，它就退化成 $O(n^3)$ 。

G.3 为什么小例子会让人觉得几乎没成本

回到[附录 C.1](#)中的匀速跟踪例子。那里 $n = 2$, $m = 1$ 。于是：

- 协方差存储只要 $2^2 = 4$ 个数；
- 三次复杂度量级只有 $2^3 = 8$ ；
- 创新协方差甚至只是一个标量。

这就是为什么教材中的卡尔曼滤波看上去如此轻松。理论并没有骗人，只是尺度太小，以至于计算负担完全看不见。